ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 11

Управление доступом на основе ролей (RBAC)

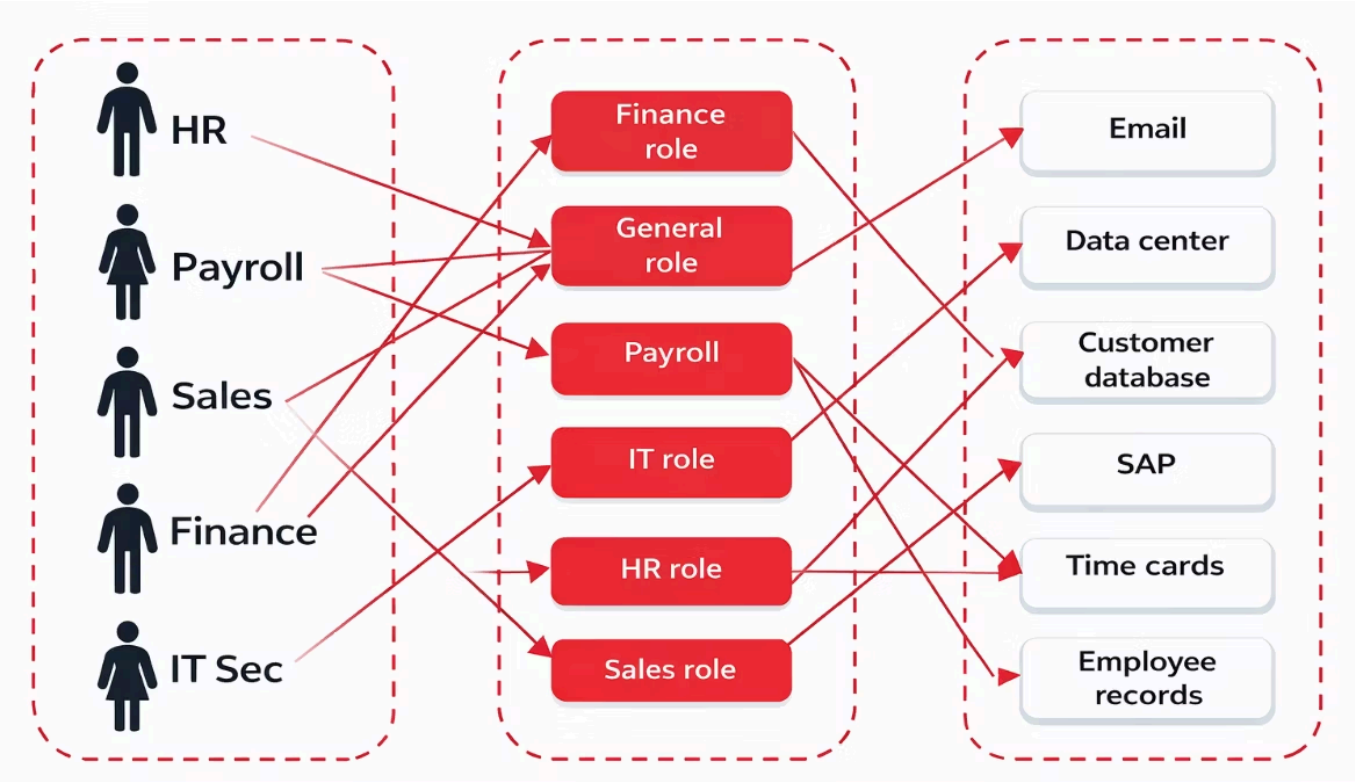
Рассмотрим модель аутентификации и авторизации пользователей в веб-приложениях, основанную на системе ролей. Решение практического задания осуществляется внутри соответствующей рабочей тетради, расположенной в СДО.

RBAC (role-based access control)

RBAC — это модель контроля доступа, при которой права пользователей к системным ресурсам предоставляются через роли. При этом роль представляет собой набор разрешений, необходимых для выполнения определенного вида задач.

Вместо того чтобы выдавать каждому сотруднику права вручную, администратор назначает пользователям роли, а доступ к ресурсам определяется уже через связанные с ними разрешения. Такой подход упрощает управление доступом и позволяет масштабировать систему по мере роста компании и изменения ее структуры.

Условно модель можно визуализировать следующим образом:



Представим, что система имеет 3 роли: администратор, продавец, пользователь. Тогда пользователь сможет только просматривать товары и совершать покупки, продавец сможет управлять товарами, а администратор - управлять товарами и пользователями.

Реализация системы ролей на сервере

Для реализации RBAC на сервере создадим два middleware: authMiddleware - для проверки аутентификации пользователя и rolesMiddleware - для ограничения доступа к эндпоинтам для конкретных ролей.

```
const express = require("express");
const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");

const app = express();
app.use(express.json());

const PORT = 3000;

// Секреты подписи
const ACCESS_SECRET = "access_secret";
const REFRESH_SECRET = "refresh_secret";

// Время жизни токенов
const ACCESS_EXPIRES_IN = "15m";
const REFRESH_EXPIRES_IN = "7d";
```

```
// { id, username, passwordHash, role }
const users = [];

// Хранилище refresh-токенов
const refreshTokens = new Set();

function generateAccessToken(user) {
  return jwt.sign(
    {
      sub: user.id,
      username: user.username,
      role: user.role
    },
    ACCESS_SECRET,
    {
      expiresIn: ACCESS_EXPIRES_IN,
    }
  );
}

function generateRefreshToken(user) {
  return jwt.sign(
    {
      sub: user.id,
      username: user.username,
      role: user.role
    },
    REFRESH_SECRET,
    {
      expiresIn: REFRESH_EXPIRES_IN,
    }
  );
}

// Auth middleware
function authMiddleware(req, res, next) {
  const header = req.headers.authorization || "";
  const [scheme, token] = header.split(" ");

  if (scheme !== "Bearer" || !token) {
    return res.status(401).json({
      error: "Missing or invalid Authorization header",
    });
  }
```

```
  try {
    const payload = jwt.verify(token, ACCESS_SECRET);
    req.user = payload;
    next();
  } catch (err) {
    return res.status(401).json({
      error: "Invalid or expired token",
    });
  }
}

// Role middleware
function roleMiddleware(allowedRoles) {
  return (req, res, next) => {
    if (!req.user || !allowedRoles.includes(req.user.role)) {
      return res.status(403).json({
        error: "Forbidden",
      });
    }
    next();
  };
}

app.post("/api/auth/register", async (req, res) => {
  const { username, password, role } = req.body;

  if (!username || !password) {
    return res.status(400).json({
      error: "username and password are required",
    });
  }

  const exists = users.some((u) => u.username === username);
  if (exists) {
    return res.status(409).json({
      error: "username already exists",
    });
  }

  const passwordHash = await bcrypt.hash(password, 10);

  const user = {
    id: String(users.length + 1),
    username,
    passwordHash,
    role: role || "user"
```

```
  };

  users.push(user);

  res.status(201).json({
    id: user.id,
    username: user.username,
    role: user.role
  });
});

app.post("/api/auth/login", async (req, res) => {
  const { username, password } = req.body;

  if (!username || !password) {
    return res.status(400).json({
      error: "username and password are required",
    });
  }

  const user = users.find((u) => u.username === username);
  if (!user) {
    return res.status(401).json({
      error: "Invalid credentials",
    });
  }

  const isValid = await bcrypt.compare(password, user.passwordHash);
  if (!isValid) {
    return res.status(401).json({
      error: "Invalid credentials",
    });
  }

  const accessToken = generateAccessToken(user);
  const refreshToken = generateRefreshToken(user);

  refreshTokens.add(refreshToken);

  res.json({
    accessToken,
    refreshToken,
  });
});

app.post("/api/auth/refresh", (req, res) => {
```

```
  const { refreshToken } = req.body;

  if (!refreshToken) {
    return res.status(400).json({
      error: "refreshToken is required",
    });
  }

  if (!refreshTokens.has(refreshToken)) {
    return res.status(401).json({
      error: "Invalid refresh token",
    });
  }

  try {
    const payload = jwt.verify(refreshToken, REFRESH_SECRET);

    const user = users.find((u) => u.id === payload.sub);
    if (!user) {
      return res.status(401).json({
        error: "User not found",
      });
    }

    refreshTokens.delete(refreshToken);

    const newAccessToken = generateAccessToken(user);
    const newRefreshToken = generateRefreshToken(user);

    refreshTokens.add(newRefreshToken);

    res.json({
      accessToken: newAccessToken,
      refreshToken: newRefreshToken,
    });
  } catch (err) {
    return res.status(401).json({
      error: "Invalid or expired refresh token",
    });
  }
});

// доступ продавцу и админу
app.get("/api/protected-route",
    authMiddleware, roleMiddleware(["seller", "admin"]),
    (req, res) => {
```

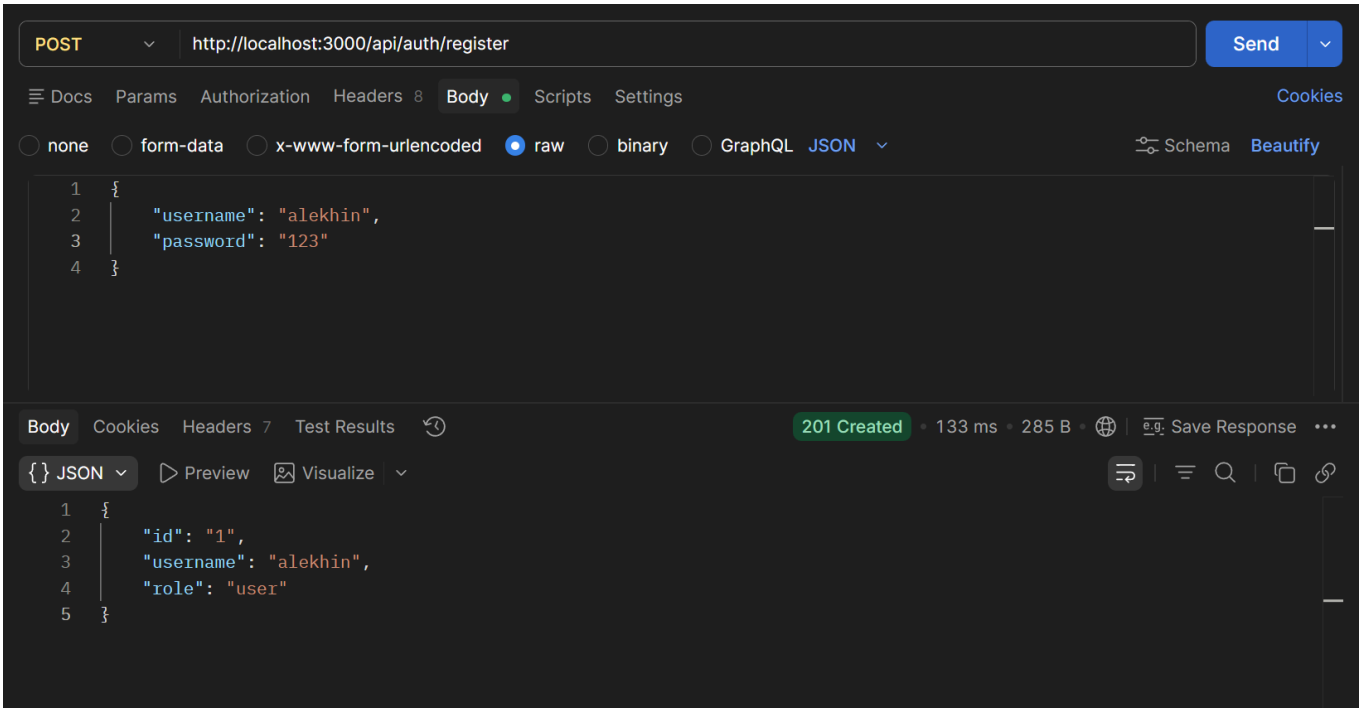
```
        res.json({
            message: "Protected route for seller or admin",
            user: req.user
        });
    }
);

// доступ только админу
app.get("/api/protected-admin-route",
    authMiddleware, roleMiddleware(["admin"]),
    (req, res) => {
        res.json({
            message: "Admin only route",
            user: req.user
        });
    }
);

app.listen(PORT, () => {
  console.log(`Сервер запущен на http://localhost:${PORT}`);
});
```

Таким образом, `/api/protected-route` доступен всем ролям, кроме пользователя, а `/api/protected-admin-route` - только администратору. Протестируем.

Зарегистрируем пользователя



Войдем в аккаунт

POST http://localhost:3000/api/auth/login Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
{
    "username": "alekhin",
    "password": "123"
}
```

Body Cookies Headers 7 Test Results 200 OK 151 ms 649 B

{} JSON Preview Visualize


```
{
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJzdWIiOiIxIiwidXNlcm5hbWUiOiJhbGVraGluIiwicm9sZSI6InVzZXIiLCJpYXQiOjE3NzMxMjYzOTQsImV4cCI6MTc3MzEyNzI5NH0.
        wd9RrYv6Gla9r1YSsv70BFSF2W5uXTzuujT1ZuKdh5I",
    "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJzdWIiOiIxIiwidXNlcm5hbWUiOiJhbGVraGluIiwicm9sZSI6InVzZXIiLCJpYXQiOjE3NzMxMjYzOTQsImV4cCI6MTc3MzczMTE5NH0.
        -NzbAHPE47R8r_K0om0Z1GWUMmmU-NzFSBARKSO-elc"
}
```


Обратимся к `/api/protected-route`

GET http://localhost:3000/api/protected-route Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Auth Type

Bearer Token

Token ••••••••••••••••••••••••••••••••••••

The authorization header will be automatically generated when you send the request. Learn more about Bearer Token authorization.

Body Cookies Headers 7 Test Results 403 Forbidden 20 ms 263 B

{} JSON Preview Try with different auth credentials

```
{
    "error": "Forbidden"
}
```

Зарегистрируем продавца

POST http://localhost:3000/api/auth/register Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
{
    "username": "sel",
    "password": "123",
    "role": "seller"
}
```

Body Cookies Headers 7 Test Results 201 Created 202 ms 283 B Save Response

{} JSON Preview Visualize

```
{
    "id": "2",
    "username": "sel",
    "role": "seller"
}
```

Войдем в аккаунт

POST http://localhost:3000/api/auth/login Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

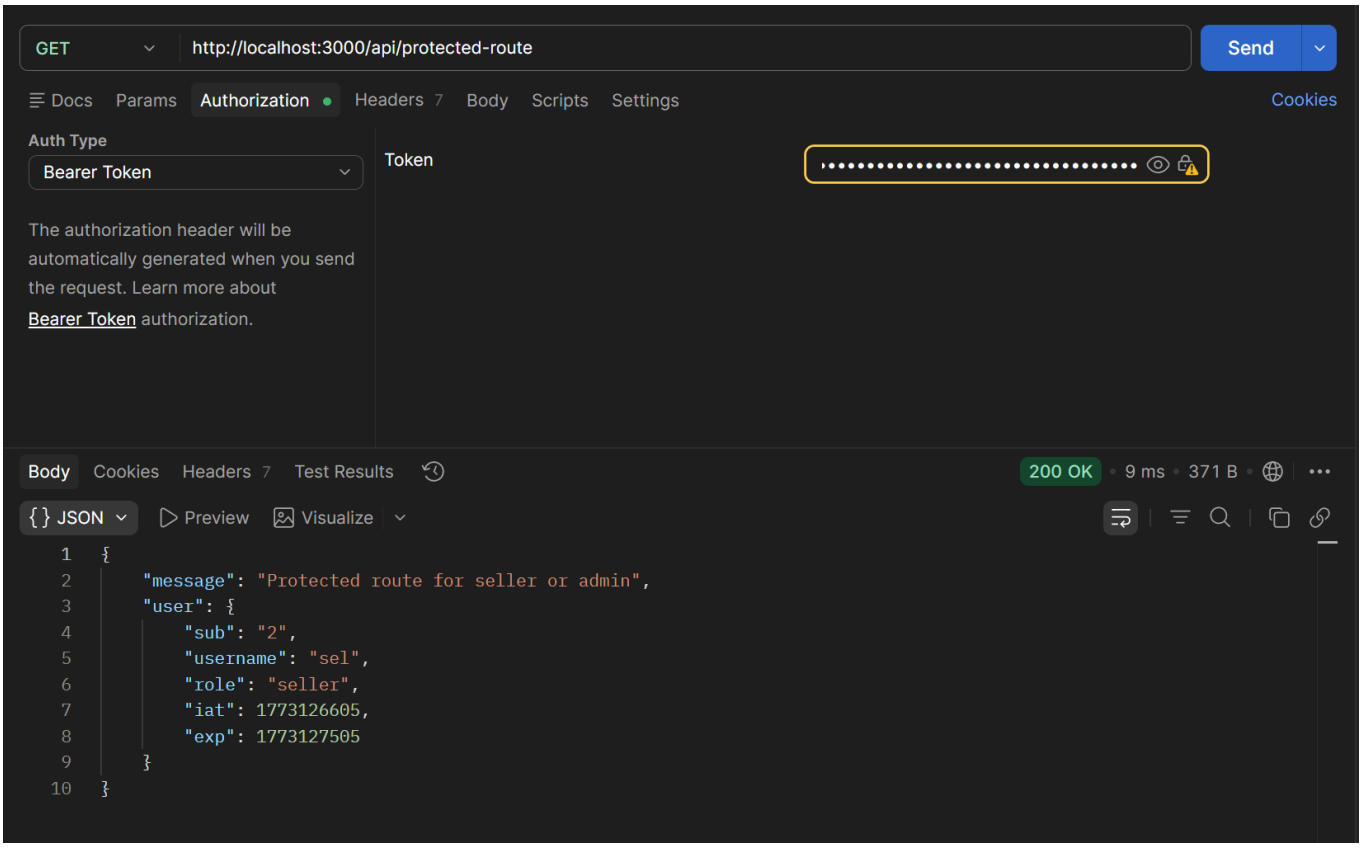
```
{
    "username": "sel",
    "password": "123"
}
```

Body Cookies Headers 7 Test Results 200 OK 155 ms 643 B

{} JSON Preview Visualize


```
{
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJzdWIiOiIyIiwidXNlcm5hbWUiOiJzZWwiLCJyb2xlIjoic2VsbGVyIiwiaWF0IjoxNzczMTI2NjA1LCJleHAiOjE3NzMxMjc1MDV9.
        TY4nQ3yGsAnrDl-a4--xIiDfZH4vxs5gyOcWgGeZkdI",
    "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJzdWIiOiIyIiwidXNlcm5hbWUiOiJzZWwiLCJyb2xlIjoic2VsbGVyIiwiaWF0IjoxNzczMTI2NjA1LCJleHAiOjE3NzM3MzE0MDV9.
        5U9iJ-qBELSEgajfnB9mSuolLfUzeHtDPS3Z28phhkA"
}
```


Обратимся к `/api/protected-route`



Таким образом маршруты становятся доступными только тем ролям, которые указаны в `roleMiddleware`

Практическое задание

Необходимо доработать задание из практического занятия №10: добавьте систему ролей, реализующих систему прав доступа:

- Гость - не аутентифицированный пользователь
- Пользователь - пользователь сайта (только просмотр товаров)
- Продавец - сотрудник сайта (добавление и редактирование товаров)
- Администратор - управленец сайта (права продавца + управление пользователями)

Минимальный список маршрутов приложения, а также более детальное описания прав доступа для каждой роли и маршрута приведено в таблице:

Маршрут	Метод	Доступ	Описание
/api/auth/register	POST	Гость	Регистрация (создание) пользователя
/api/auth/login	POST	Гость	Вход в систему
/api/auth/refresh	POST	Гость	Обновление пары токенов
/api/auth/me	GET	Пользователь	Получение текущего пользователя
/api/users	GET	Администратор	Получить список пользователей

Маршрут	Метод	Доступ	Описание
/api/users/:id	GET	Администратор	Получить пользователя по id
/api/users/:id	PUT	Администратор	Обновить информацию пользователя
/api/users/:id	DELETE	Администратор	Заблокировать пользователя
/api/products	POST	Продавец	Создать товар
/api/products	GET	Пользователь	Получить список товаров
/api/products/:id	GET	Пользователь	Получить товар по id
/api/products/:id	PUT	Продавец	Обновить параметры товара
/api/products/:id	DELETE	Администратор	Удалить товар

Клиентская часть (фронтенд) должна позволять управлять:

- товарами: просматривать список, создавать новый товар, просматривать детальную информацию о товаре по id, обновлять товар по id, удалять товар по id;
- пользователями: администратор системы должен иметь возможность просматривать список пользователей, обновлять их информацию и блокировать их.

Формат отчета

В качестве ответа на задание необходимо прикрепить ссылку на репозиторий с реализованной практикой. Ссылка подгружается в соответствующий раздел СДО: Задания для самостоятельной работы -> Практические занятия 11-12.